

Operating Systems

Project Checkpoint 3

112070002 張博安

Table of Contents

1	Screenshots for Compilation	2
2	Screenshots and Explanation	2
2.1	Producer	3
2.2	Consumer	6
2.3	Final Result	10

1 Screenshots for Compilation

```
boanz@BENSONCH /cygdrive/d/os/ppc3
$ make clean
rm *.hex *.ihx *.lnk *.lst *.map *.mem *.rel *.rst *.sym *.asm *.lk
rm: cannot remove '*.hex': No such file or directory
rm: cannot remove '*.ihx': No such file or directory
rm: cannot remove '*.lnk': No such file or directory
rm: cannot remove '*.map': No such file or directory
rm: cannot remove '*.mem': No such file or directory
rm: cannot remove '*.rel': No such file or directory
rm: cannot remove '*.rst': No such file or directory
rm: cannot remove '*.lk': No such file or directory
make: *** [Makefile:25: clean] Error 1

boanz@BENSONCH /cygdrive/d/os/ppc3
$ make
sdcc -c testpreempt.c
testpreempt.c:69: warning 158: overflow in implicit constant conversion
sdcc -c preemptive.c
preemptive.c:164: warning 85: in function ThreadCreate unreferenced function argument : 'fp'
preemptive.c:205: warning 158: overflow in implicit constant conversion
sdcc -o testpreempt.hex testpreempt.rel preemptive.rel
```

Figure 1: Compilation result of `make clean` and `make`

The `make clean` output shows multiple `No such file or directory` messages and `Error 1` because no compiled files exist yet. This is expected behavior and not an actual error.

The `make` output shows that SDCC successfully compiles `testpreempt.c` and `preemptive.c`, generating `testpreempt.hex`. Three non-fatal warnings appear during compilation. First, `testpreempt.c` line 69 produces an overflow warning for the implicit constant conversion of `TH1 = -6`, which is the standard 8051 baud rate convention. Second, `preemptive.c` line 164 reports that the function argument `fp` in `ThreadCreate()` is unreferenced, because `fp` is accessed through `DPTR` in inline assembly rather than directly in C. Third, `preemptive.c` line 205 produces another overflow warning for the implicit constant conversion of `-1` in `ThreadExit()`. None of these affect program execution.

2 Screenshots and Explanation

	value	Global	Global Defined In Module
C:	00000081	_Producer	testpreempt
C:	000000b2	_Consumer	testpreempt
C:	00000123	_main	testpreempt
C:	00000135	__sdcc_gsinit_startup	testpreempt
C:	00000139	__mcs51_genRAMCLEAR	testpreempt
C:	0000013A	__mcs51_genXINIT	testpreempt
C:	0000013B	__mcs51_genXRAMCLEAR	testpreempt
C:	0000013C	_timer0_ISR	testpreempt
C:	00000142	_Bootstrap	preemptive
C:	00000168	_myTimer0Handler	preemptive
C:	000001BD	_ThreadCreate	preemptive
C:	00000234	_ThreadYield	preemptive
C:	0000028F	_ThreadExit	preemptive

Figure 2: Function address list from `testpreempt.map`

The memory addresses of all functions are shown in the `.map` file. The semaphore and shared variable addresses are as follows: `mutex` (0x34) is initialised to 1, `empty` (0x35) to

3, and `full` (0x36) to 0. The 3-slot circular buffer `circBuf` occupies addresses 0x30–0x32, `writeCursor` is at 0x37, and `readCursor` is at 0x38.

2.1 Producer

The `Producer`'s code address range is 0x0081 to 0x00D1. The `Producer` writes characters 'A'–'Z' into the circular buffer and operates on semaphores before and after each write.

`empty` counts the number of free slots in the buffer. It decreases by 1 when the `Producer` writes a character, and increases by 1 when the `Consumer` consumes one. `full` counts the number of slots containing valid data. It increases by 1 when the `Producer` writes, and decreases by 1 when the `Consumer` reads.

```
#define SemaphoreWaitBody(S, label) \
{ \
    __asm \
    label: \
    MOV A, CNAME(S) \
    JZ label \
    JB ACC.7, label \
    DEC CNAME(S) \
    __endasm; \
}
```

Figure 3: Screenshot of `SemaphoreWaitBody` code

The `SemaphoreWaitBody` macro implements busy-wait in inline assembly. A unique `label` is passed on each call to serve as the loop target. The logic is: read the semaphore value into ACC; if ACC = 0 (exhausted), jump back to wait; if ACC.7 = 1 (negative), jump back to wait; otherwise fall through and execute `DEC CNAME(S)` to decrement the semaphore, completing the Wait operation.

```
#define SemaphoreSignal(s) \
{ \
    __asm \
    INC CNAME(s) \
    __endasm; \
}
```

Figure 4: Screenshot of `SemaphoreSignal` code

The `SemaphoreSignal` macro uses a single 8051 assembly instruction `INC CNAME(s)` to atomically increment the semaphore value. Since `INC` is a single instruction, it is inherently atomic and can be safely called from any thread without disabling interrupts.

```

void Producer(void)
{
    nextChar    = 'A';
    writeCursor = 0;

    while (1) {
        /* wait for a free slot, then acquire the lock */
        SemaphoreWait(empty);
        SemaphoreWait(mutex);

        /* --- critical section --- */
        circBuf[writeCursor] = nextChar;
        writeCursor = (writeCursor == 2) ? 0 : writeCursor + 1;
        /* --- end critical section --- */

        SemaphoreSignal(mutex);
        SemaphoreSignal(full);

        /* advance character outside the critical section */
        nextChar = (nextChar == 'Z') ? 'A' : nextChar + 1;
    }
}

```

Figure 5: Screenshot of `Producer` code

The `Producer` code is shown above. The main loop structure is: `SemaphoreWait(empty)` waits for a free slot to ensure the buffer is not full; `SemaphoreWait(mutex)` acquires the mutual exclusion lock to enter the critical section; the character `nextChar` is written to `circBuf[writeCursor]` and `writeCursor` is advanced; `SemaphoreSignal(mutex)` releases the lock; `SemaphoreSignal(full)` notifies the `Consumer` that new data is available; finally, `nextChar` is incremented outside the critical section (cycling 'A' → 'Z' → 'A'). The critical section contains only the buffer write and cursor update, keeping it as short as possible.

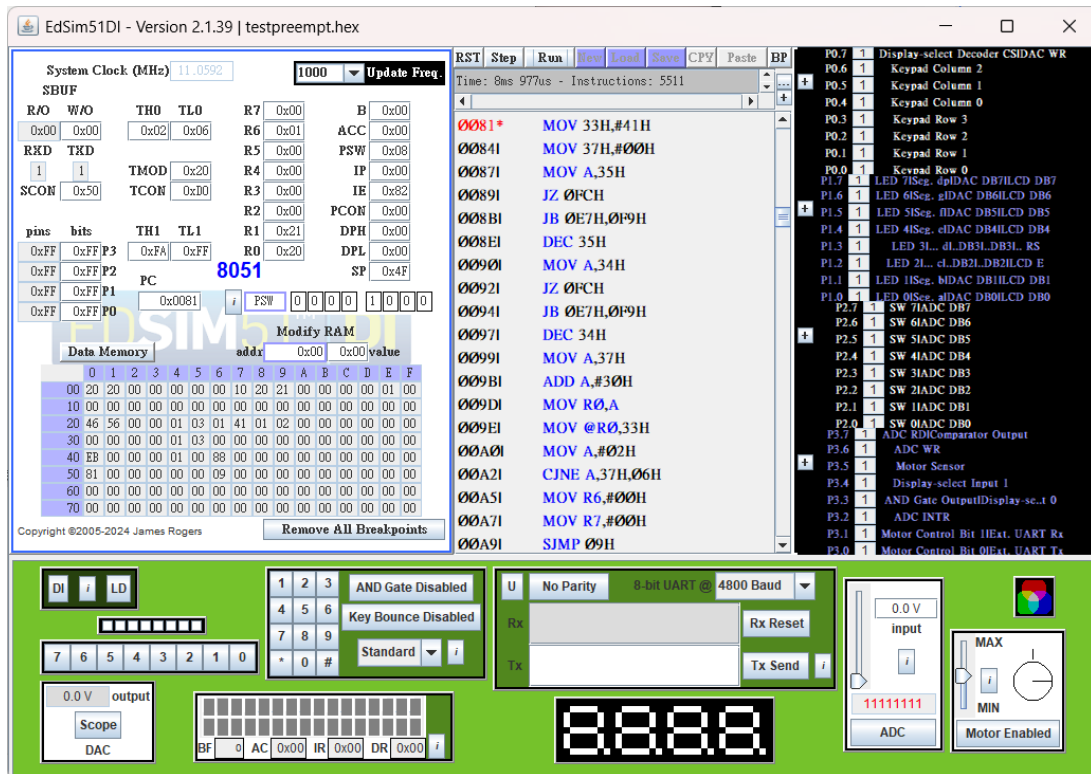


Figure 6: Screenshot 1: **Producer** enters, initial semaphore values

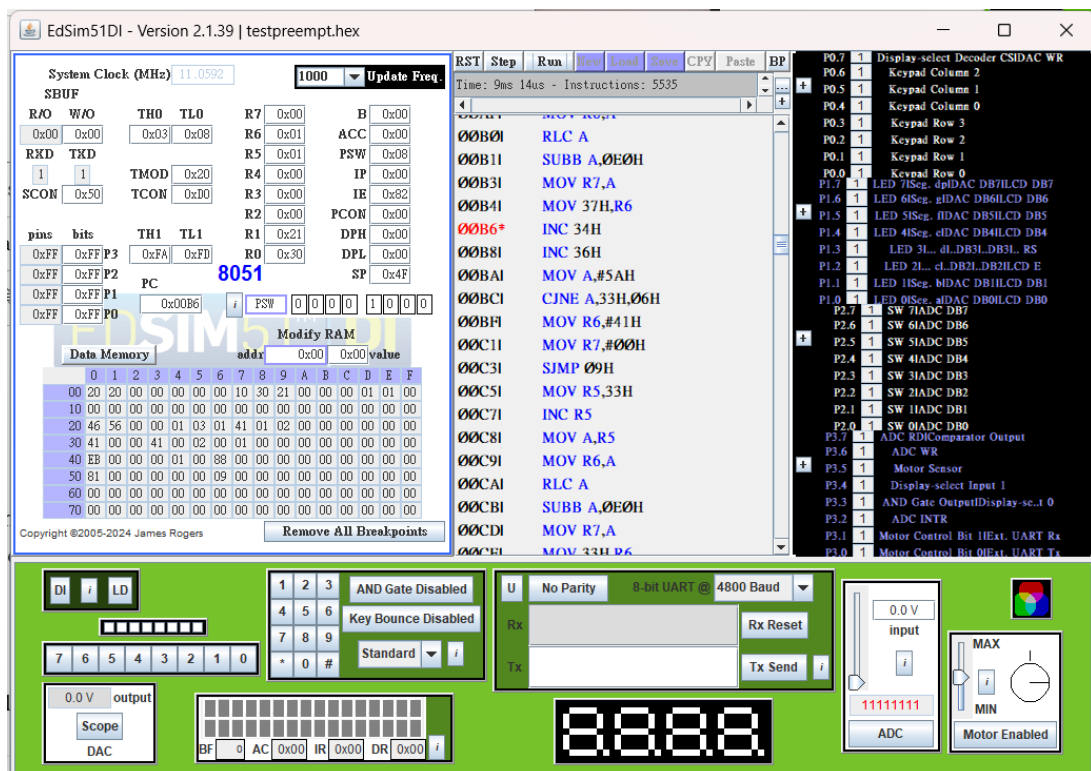


Figure 7: Screenshot 2: after writing to buffer, before Signal

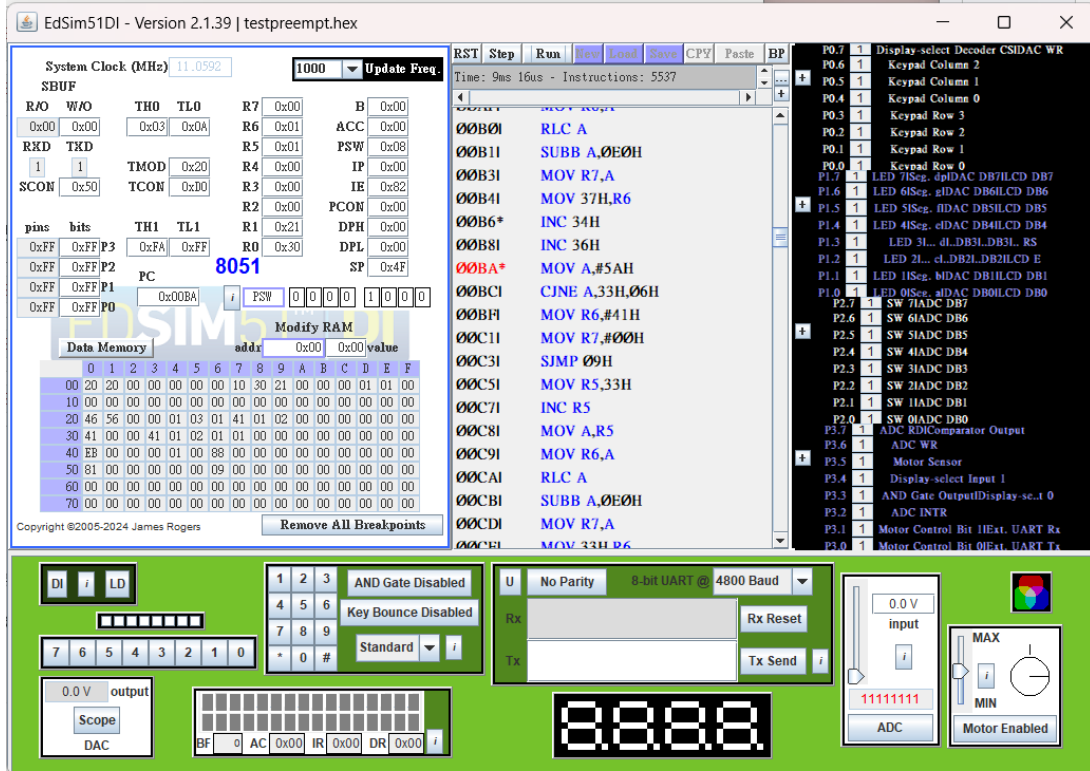


Figure 8: Screenshot 3: after both Signals complete

Screenshot 1 (Producer enters, initial semaphore values): PC is stopped at 0x0081 (first instruction of **Producer**). Semaphores are at their initial values: mutex = 1 (available), empty = 3 (all slots free), full = 0 (no data in buffer).

Screenshot 2 (after writing to buffer, before Signal): PC is stopped at 0x00B6 (INC 34H, the first INC of **SemaphoreSignal(mutex)**). At this point: mutex = 0 (**SemaphoreWait(mutex)** has executed, lock is held), empty = 2 (**SemaphoreWait(empty)** has executed, decremented from 3 to 2), full = 0 (**SemaphoreSignal(full)** has not yet executed). **circBuf[0]** at 0x30 = 0x41 ('A'), confirming the character has been written.

Screenshot 3 (after both Signals complete): PC is stopped at 0x00BA (after both Signal instructions). At this point: mutex = 1 (**SemaphoreSignal(mutex)** complete, lock released), empty = 2 (unchanged), full = 1 (**SemaphoreSignal(full)** complete, incremented from 0 to 1). The semaphore changes confirm that the **Producer** successfully wrote one character and correctly released the semaphores.

2.2 Consumer

The **Consumer**'s code address range is 0x00D2 to 0x0122. The **Consumer** retrieves characters from the circular buffer and transmits them via the UART serial port.

```

void Consumer(void)
{
    /* initialise UART for 4800 baud (Timer 1, mode 2) */
    __critical {
        TMOD |= 0x20;
        TH1 = -6;
        SCON = 0x50;
        TR1 = 1;
    }
    readCursor = 0;

    while (1) {
        /* wait for a filled slot, then acquire the lock */
        SemaphoreWait(full);
        SemaphoreWait(mutex);

        /* critical section */
        SBUF = circBuf[readCursor];
        readCursor = (readCursor == 2) ? 0 : readCursor + 1;
        /* end critical section */

        SemaphoreSignal(mutex);
        SemaphoreSignal(empty);

        /* wait for UART transmit to complete outside critical section */
        while (!TI);
        TI = 0;
    }
}

```

Figure 9: Screenshot of `Consumer` code

The `Consumer` code is shown above. The function begins by initialising the UART inside `__critical{}` (Timer 1 mode 2, 4800 baud) to prevent interruption during setup. The main loop structure is: `SemaphoreWait(full)` waits for valid data to ensure the buffer is not empty; `SemaphoreWait(mutex)` acquires the lock to enter the critical section; the character at `circBuf[readCursor]` is written to SBUF and `readCursor` is advanced; `SemaphoreSignal(mutex)` releases the lock; `SemaphoreSignal(empty)` notifies the `Producer` that a slot is free; finally, the UART transmission completion is waited for outside the critical section (`while(!TI)`), and the TI flag is cleared manually.

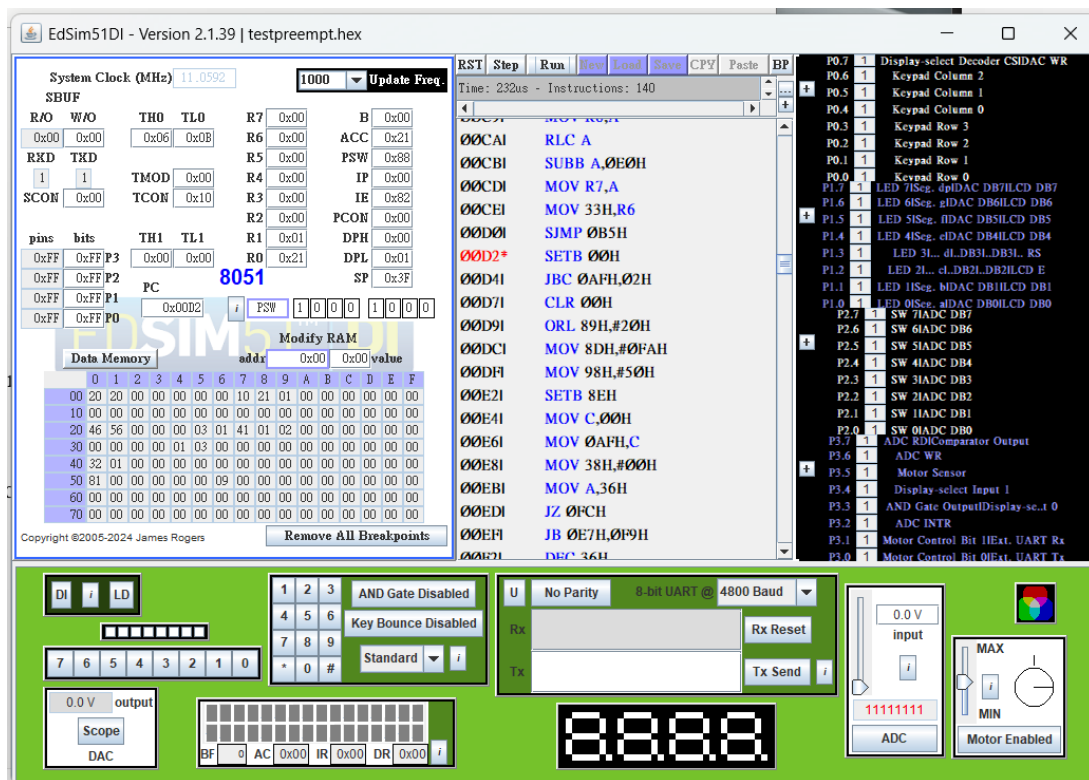


Figure 10: Screenshot 1: **Consumer** enters for the first time, blocked

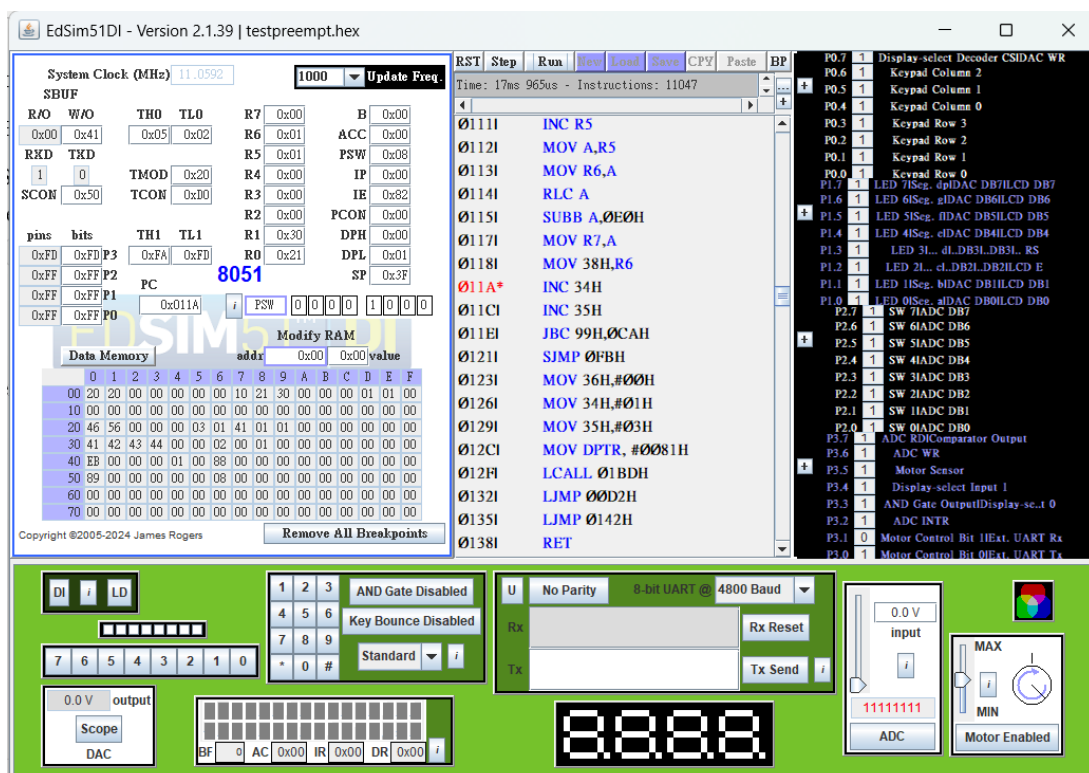


Figure 11: Screenshot 2: **Consumer** enters for the second time, after read, before Signal

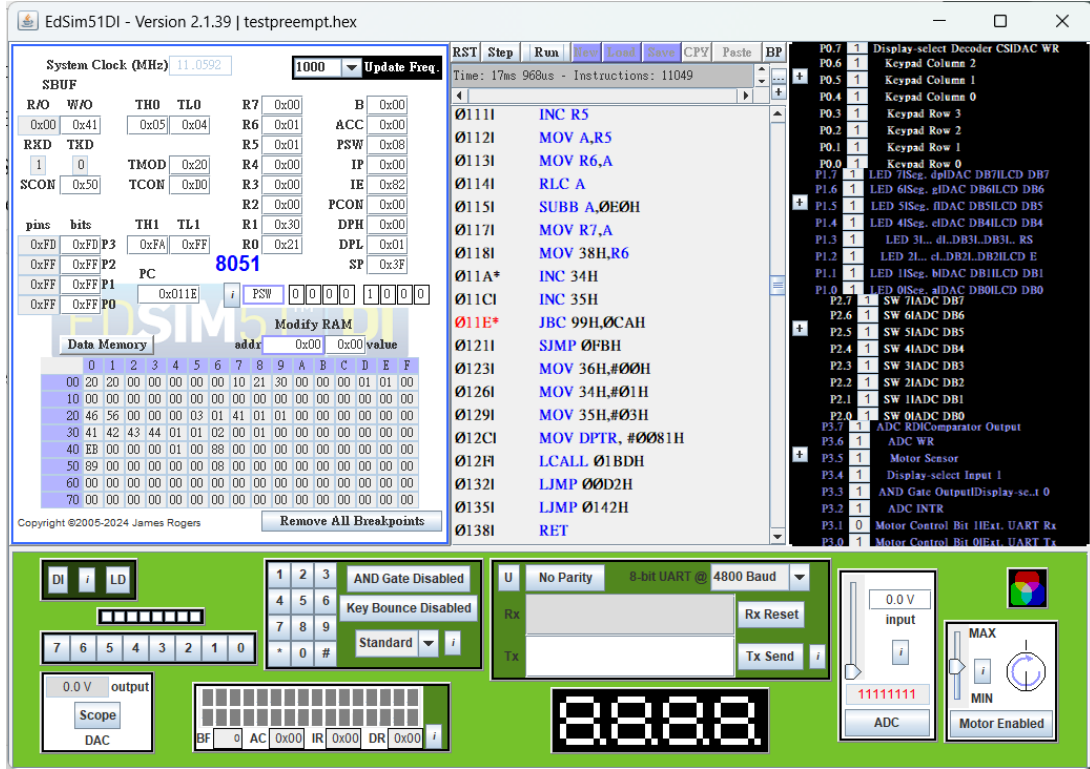


Figure 12: Screenshot 3: after both Signals complete

Screenshot 1 (Consumer enters for the first time, blocked): PC is stopped at 0x00D2 (first instruction of **Consumer**). Semaphores are at initial values: mutex = 1, empty = 3, full = 0. Since full = 0, the **Consumer** is blocked at the busy-wait loop inside **SemaphoreWait(full)**, waiting for the **Producer** to write data. It is subsequently preempted by Timer 0 and the **Producer** runs.

Screenshot 2 (Consumer enters for the second time, after read, before Signal): After the **Producer** runs three times and fills the buffer (empty: 3 → 0, full: 0 → 3), the **Consumer** is scheduled again and successfully passes both **SemaphoreWait** calls. PC is stopped at 0x011A (INC 34H, just before **SemaphoreSignal(mutex)**): mutex = 0 (lock is held, inside critical section), empty = 0 (buffer was filled by **Producer**, **Consumer** has not yet signalled), full = 2 (**SemaphoreWait(full)** has executed, decremented from 3 to 2). SBUF has received the first character 'A'.

Screenshot 3 (after both Signals complete): PC is stopped at 0x011E (after both Signal instructions complete): mutex = 1 (**SemaphoreSignal(mutex)** complete, lock released), empty = 1 (**SemaphoreSignal(empty)** complete, incremented from 0 to 1), full = 2 (unchanged). At this point empty + full = 1 + 2 = 3, returning to a consistent state and confirming correct semaphore operation.

2.3 Final Result

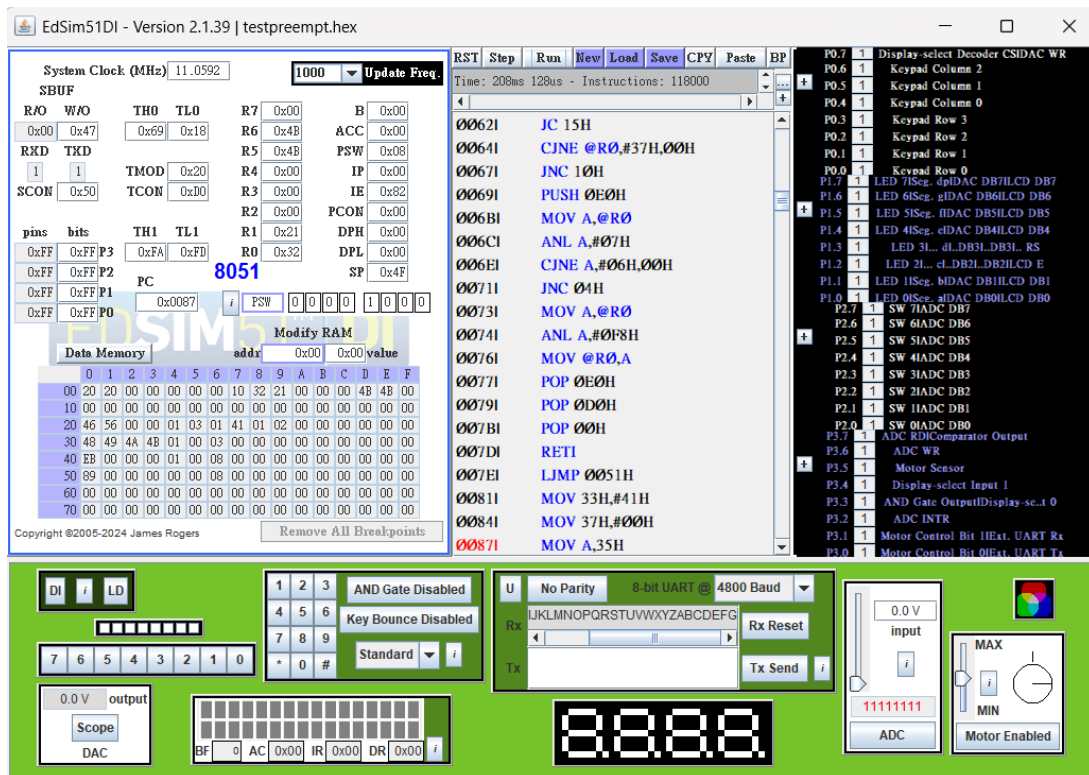


Figure 13: Screenshot after executing for some time

After running for some time, the Rx window continuously displays IJKLMNOPQRSTUVWXYZABCDEFGH..., with characters correctly cycling from 'Z' back to 'A', confirming that **Producer** and **Consumer** are alternating correctly. At this point, the buffer holds 'H', 'I', 'J' (addresses 0x30–0x32), and SBUF is transmitting 'G' (0x47). The semaphore values are mutex = 1, empty = 2, full = 1, indicating one valid character remains in the buffer awaiting consumption. The overall result is correct, and both the preemptive scheduling and semaphore synchronization are functioning as intended.